EcoWatch AI — Master Reference Prompt

> Copy and use this anytime. Everything you need in one place.

🧭 Project in One Line

Build a deep learning system that uses **Sentinel-2 satellite imagery** + **KGIS industrial boundary shapefiles** to automatically detect green belt violations and unauthorized construction within factory premises in Karnataka, and generate a compliance report with annotated visual evidence.

📍 Location Selection

Primary Study Area (Recommended — use all 3 for a strong paper)

Location	Why
Peenya Industrial Area, Bengaluru	Largest industrial estate in South Asia — dense factory count, well-documented boundary disputes, long Sentinel-2 archive
Bommasandra Industrial Area, Bengaluru	Active construction zone near lake buffers — high chance of real violations visible in imagery
Nanjangud Industrial Area, Mysuru	Mix of pharma and chemical factories — CPCB-monitored, green belt compliance legally mandated

Why these work for your project

- All three are in Karnataka → KGIS data directly applies
- All three have Sentinel-2 coverage from 2017 onwards → enough temporal depth for before/after analysis
- All three have documented CPCB compliance histories → ground truth can be partially verified from public inspection reports
- Urban-industrial fringe means clear contrast between vegetation and built-up land → easier model training

Coordinates (for GEE Area of Interest)

```javascript
// Peenya Industrial Area
var peenya = ee.Geometry.Rectangle([77.51, 13.02, 77.58, 13.07]);

// Bommasandra Industrial Area
var bommasandra = ee.Geometry.Rectangle([77.66, 12.78, 77.73, 12.83]);

// Nanjangud Industrial Area
var nanjangud = ee.Geometry.Rectangle([76.67, 12.08, 76.74, 12.13]);
```

📦 Data Collection

1. KGIS Data — Karnataka GIS Portal

URL: https://kgis.ksrsac.in/kgis/

What to download:

- Industrial area boundary shapefiles
- Land use / land cover (LULC) layers
- Water body and lake buffer layers (for cross-validation)
- Revenue / survey number cadastral layers

How to use:

- Open portal → select Karnataka district → download as shapefile (.shp)
- Load into QGIS → reproject to EPSG:32643 (WGS 84 / UTM zone 43N — standard for Karnataka)
- Clip to your AOI (Peenya / Bommasandra / Nanjangud)
- Export as GeoJSON for use in Python with GeoPandas

---

2. Sentinel-2 Data — via Google Earth Engine (Preferred)

Why Sentinel-2 over Landsat:

- 10m spatial resolution (Landsat = 30m) — critical for detecting small green belt changes within factory premises

- 5-day revisit time (Landsat = 16 days) — more image options, easier to find cloud-free pairs
- 13 spectral bands including red-edge — better vegetation discrimination

GEE Code to pull Sentinel-2:

```javascript
```

```
// ── Sentinel-2 SR — Peenya, cloud-filtered ──────────────────────────
var aoi = ee.Geometry.Rectangle([77.51, 13.02, 77.58, 13.07]);

// Baseline period (T1) — before
var s2_t1 = ee.ImageCollection('COPERNICUS/S2_SR_HARMONIZED')
  .filterBounds(aoi)
  .filterDate('2019-01-01', '2019-12-31')
  .filter(ee.Filter.lt('CLOUDY_PIXEL_PERCENTAGE', 10))
  .median()
  .clip(aoi);

// Recent period (T2) — after
var s2_t2 = ee.ImageCollection('COPERNICUS/S2_SR_HARMONIZED')
  .filterBounds(aoi)
  .filterDate('2023-01-01', '2023-12-31')
  .filter(ee.Filter.lt('CLOUDY_PIXEL_PERCENTAGE', 10))
  .median()
  .clip(aoi);

// Compute NDVI for both periods
var ndvi_t1 = s2_t1.normalizedDifference(['B8', 'B4']).rename('NDVI_T1');
var ndvi_t2 = s2_t2.normalizedDifference(['B8', 'B4']).rename('NDVI_T2');

// NDVI difference (negative = vegetation loss)
var ndvi_diff = ndvi_t2.subtract(ndvi_t1).rename('NDVI_DIFF');

// Stack bands for export: RGB + NIR + NDVI
var export_t1 = s2_t1.select(['B4','B3','B2','B8']).addBands(ndvi_t1);
var export_t2 = s2_t2.select(['B4','B3','B2','B8']).addBands(ndvi_t2);

// Export to Google Drive
Export.image.toDrive({
  image: export_t1,
  description: 'Peenya_S2_T1_2019',
  folder: 'EcoWatch_Data',
  region: aoi,
  scale: 10,
  crs: 'EPSG:32643',
  maxPixels: 1e10
});

Export.image.toDrive({
  image: export_t2,
```

```
  description: 'Peenya_S2_T2_2023',
  folder: 'EcoWatch_Data',
  region: aoi,
  scale: 10,
  crs: 'EPSG:32643',
  maxPixels: 1e10
});

Export.image.toDrive({
  image: ndvi_diff,
  description: 'Peenya_NDVI_DIFF',
  folder: 'EcoWatch_Data',
  region: aoi,
  scale: 10,
  crs: 'EPSG:32643',
  maxPixels: 1e10
});
```

Bands to use:

Band	Name	Resolution	Use
B2	Blue	10m	RGB composite
B3	Green	10m	RGB composite
B4	Red	10m	RGB composite, NDVI
B8	NIR	10m	NDVI, vegetation detection
B11	SWIR-1	20m	Built-up detection (NDBI)
B12	SWIR-2	20m	Bare soil, construction

Indices to compute:

```python
```

```
# NDVI — Vegetation health (positive = vegetated)
NDVI = (B8 - B4) / (B8 + B4)

# NDBI — Built-up index (positive = built-up/construction)
NDBI = (B11 - B8) / (B11 + B8)

# SAVI — Soil-adjusted vegetation (better than NDVI in sparse cover)
SAVI = ((B8 - B4) / (B8 + B4 + 0.5)) * 1.5
```

---

3. Landsat Data — via GEE (Backup / Long archive)

When to use Landsat instead of / alongside Sentinel-2:

- You need imagery before 2017 (Sentinel-2 archive starts 2017; Landsat goes back to 1972)
- You want a longer historical baseline (e.g. 2013 vs 2024)
- Cross-validation of Sentinel-2 predictions

GEE Code — Landsat 8 OLI:

```javascript
```

```
var aoi = ee.Geometry.Rectangle([77.51, 13.02, 77.58, 13.07]);

// Landsat 8 Surface Reflectance
var l8_t1 = ee.ImageCollection('LANDSAT/LC08/C02/T1_L2')
  .filterBounds(aoi)
  .filterDate('2014-01-01', '2014-12-31')
  .filter(ee.Filter.lt('CLOUD_COVER', 10))
  .median()
  .clip(aoi);

// Scale bands to reflectance (Landsat Collection 2 requires this)
function applyScaleFactors(image) {
  var opticalBands = image.select('SR_B.').multiply(0.0000275).add(-0.2);
  return image.addBands(opticalBands, null, true);
}
var l8_t1 = applyScaleFactors(l8_t1);

// NDVI from Landsat 8 (B5=NIR, B4=Red)
var ndvi_l8 = l8_t1.normalizedDifference(['SR_B5', 'SR_B4']).rename('NDVI_L8');

Export.image.toDrive({
  image: l8_t1.select(['SR_B4','SR_B3','SR_B2','SR_B5']).addBands(ndvi_l8),
  description: 'Peenya_L8_T1_2014',
  folder: 'EcoWatch_Data',
  region: aoi,
  scale: 30,
  crs: 'EPSG:32643',
  maxPixels: 1e10
});
```

🧠 Models — What, Why, and How

Model 1 — Vegetation Segmentation: SegFormer-B2

What it does: Pixel-wise classification of each 10m pixel as vegetation or non-vegetation within factory boundaries.

Why SegFormer-B2 (not B4):

- B2 is lighter → trainable on CPU/low-end GPU for a course project
- Still achieves state-of-the-art mIoU on remote sensing datasets

- Transformer-based → better at capturing long-range spatial dependencies than CNN-only models

Input: 512×512 patch with 5 channels — [R, G, B, NIR, NDVI]

Output: Binary mask — vegetation (1) vs. non-vegetation (0)

Pretrained weights: `nvidia/mit-b2` from HuggingFace (ImageNet pretrained)

Fine-tuning strategy:

- Freeze encoder for first 10 epochs
- Unfreeze and fine-tune all layers for remaining epochs
- Use NDVI thresholding (NDVI > 0.3 = vegetation) to auto-generate weak labels if manual annotation is limited

Loss function: Dice Loss + Binary Cross Entropy (combined)

```python
loss = 0.5 * dice_loss(pred, target) + 0.5 * bce_loss(pred, target)
```

Library: `segmentation-models-pytorch`

```python
import segmentation_models_pytorch as smp
model = smp.Segformer(
    encoder_name="mit_b2",
    encoder_weights="imagenet",
    in_channels=5,          # R, G, B, NIR, NDVI
    classes=1,
    activation="sigmoid"
)
```

Model 2 — Change Detection: Siamese U-Net

What it does: Takes two co-registered image patches from different dates (T1, T2) and outputs a binary mask of pixels that changed significantly (vegetation loss or new construction).

Why Siamese U-Net (not BIT for course project):

- BIT (Binary change detection Transformer) is more accurate but heavier

- Siamese U-Net is well-documented, easier to implement from scratch, and sufficient for a course project
- Can be upgraded to BIT in Phase 2

Architecture:

```
T1 patch (512×512×5) ──→ Shared Encoder (ResNet-50)──→ Feature map F1
T2 patch (512×512×5) ──→ (same weights)            ──→ Feature map F2
                                                          |
                                                  |F1 - F2| (difference)
                                                          |
                                                  U-Net Decoder
                                                          |
                                              Change mask (512×512×1)
```

Input: Two 512×512×5 patches (T1 and T2) — same location, different dates

Output: Binary change mask (1 = changed, 0 = unchanged)

Loss function: Focal Loss (handles class imbalance — changed pixels are rare)

```python
# Changed pixels are typically only 5-15% of all pixels
loss = focal_loss(pred, target, alpha=0.75, gamma=2.0)
```

Implementation:

```python
```

```python
import torch
import torch.nn as nn
import segmentation_models_pytorch as smp

class SiameseUNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.encoder = smp.encoders.get_encoder(
            "resnet50", in_channels=5, weights="imagenet"
        )
        self.decoder = smp.decoders.unet.decoder.UnetDecoder(
            encoder_channels=self.encoder.out_channels,
            decoder_channels=(256, 128, 64, 32, 16),
            n_blocks=5
        )
        self.head = nn.Conv2d(16, 1, kernel_size=1)

    def forward(self, t1, t2):
        f1 = self.encoder(t1)
        f2 = self.encoder(t2)
        diff = [torch.abs(a - b) for a, b in zip(f1, f2)]
        decoded = self.decoder(*diff)
        return torch.sigmoid(self.head(decoded))
```

Model 3 — Shared Encoder (Multi-task, Optional upgrade)

What it does: Instead of two separate encoders for segmentation and change detection, one ResNet-50 encoder is shared and both task-specific decoder heads branch off from it.

When to implement: After Models 1 and 2 individually work. This is the "impressive version" for final submission.

Benefit: 30–40% fewer parameters, shared satellite-image feature learning, better generalisation with limited labelled data.

Model 4 — NDVI Trend Anomaly: LSTM (Lightweight add-on)

What it does: Takes a time series of monthly NDVI values for a factory polygon and flags anomalous drops that fall outside expected seasonal patterns.

Input: 36-step NDVI time series (3 years × 12 months) per factory unit

Output: Anomaly score (0–1) per factory per month

Why useful: Catches gradual vegetation loss that single-date change detection might miss (e.g. slow clearing over 18 months).

```python
import torch.nn as nn

class NDVIAnomalyLSTM(nn.Module):
    def __init__(self, input_size=1, hidden_size=64, num_layers=2):
        super().__init__()
        self.lstm = nn.LSTM(input_size, hidden_size, num_layers, batch_first=True)
        self.fc = nn.Linear(hidden_size, 1)

    def forward(self, x):              # x: (batch, seq_len, 1)
        out, _ = self.lstm(x)
        return torch.sigmoid(self.fc(out[:, -1, :]))
```

📊 Evaluation Metrics — Full Reference

Tier 1 — Per-model metrics

Metric	Formula	Task	Target	Why it matters
mIoU	mean(TP / TP+FP+FN) per class	Segmentation	⩾ 0.75	Standard for pixel-wise tasks
F1-change	2×P×R / (P+R) on changed class	Change detection	⩾ 0.70	Changed pixels are rare — F1 handles imbalance
Precision	TP / (TP+FP)	Both	⩾ 0.70	False alarms waste regulator time
Recall	TP / (TP+FN)	Both	⩾ 0.75	Missing a violation is worse than a false alarm
OA (Overall Accuracy)	(TP+TN) / Total	Both	⩾ 0.85	Baseline sanity check
Cohen's Kappa	(OA - Pe) / (1 - Pe)	Both	⩾ 0.65	Accounts for chance agreement — important for imbalanced data

Metric	Formula	Task	Target	Why it matters
mAP@0.5	mean AP at IoU=0.5	Detection (Phase 2)	⩾ 0.60	Standard object detection metric

Tier 2 — Change detection specific

Metric	What it measures	How to compute
FPR on unchanged zones	How often seasonal change is misclassified as violation	Test on dry-season vs wet-season pairs with known no-change ground truth
Green cover Δ error (ha)	Accuracy of area loss estimate	Predicted hectares lost vs. QGIS-measured ground truth area
Temporal consistency	Does model give stable results across months?	Run inference on 12 monthly images of same factory, measure variance

Tier 3 — System and explainability

Metric	What it measures
GradCAM IoU	Overlap between GradCAM-highlighted region and manually annotated violation area — measures if the model "looks at the right place"
Human-verified accuracy	Show 20 report outputs to a domain expert (professor / CPCB norm reader) and measure how many violations they confirm
Seasonal F1 stability	F1 score computed separately for monsoon (Jun–Sep) and dry season (Nov–Feb) — should not differ by more than 0.10

Composite Score (Propose this in your paper)

```
Composite Score =   0.35 × F1_change
                  + 0.25 × mIoU_vegetation
                  + 0.20 × mAP_construction      ← 0 if Phase 2 not done;
redistribute to others
                  + 0.10 × GradCAM_IoU
                  + 0.10 × Seasonal_F1_stability
```

🔄 Step-by-Step Methodology

Phase 0 — Setup (Week 0)

- [ ] Create GEE account at earthengine.google.com
- [ ] Download KGIS shapefiles for Peenya, Bommasandra, Nanjangud
- [ ] Install QGIS 3.x, load KGIS shapefiles, reproject to EPSG:32643
- [ ] Set up Python environment with all dependencies

Phase 1 — Data Collection (Week 1)

- [ ] Run GEE scripts above for all 3 AOIs
- [ ] Export T1 (2019 or earlier) and T2 (2023/2024) Sentinel-2 composites
- [ ] Export NDVI difference rasters
- [ ] Download to local machine from Google Drive
- [ ] Load in QGIS for visual inspection

Phase 2 — Annotation (Week 1–2)

- [ ] In QGIS, digitise vegetation masks over 50–100 factory patches from T2 imagery
- [ ] Use NDVI threshold (NDVI > 0.3) to auto-generate weak labels, then manually correct
- [ ] Create change mask pairs: for each T1/T2 pair, mark pixels that changed
- [ ] Export annotations as GeoTIFF binary masks
- [ ] Aim for minimum 200 patch pairs for change detection, 150 patches for segmentation

Phase 3 — Preprocessing Pipeline (Week 2)

- [ ] Write `sentinel_loader.py` — load GeoTIFF, stack bands, normalise
- [ ] Write `ndvi.py` — compute NDVI, NDBI, SAVI from band arrays
- [ ] Write `gis_clip.py` — clip rasters to factory polygons using GeoPandas + Rasterio
- [ ] Write `patch_generator.py` — tile clipped rasters into 512×512 patches with 50px overlap
- [ ] Write augmentation pipeline using Albumentations (horizontal flip, rotation, colour jitter)

Phase 4 — Model Training (Week 3)

- [ ] Train SegFormer-B2 for vegetation segmentation — 40 epochs, lr=1e-4, AdamW
- [ ] Train Siamese U-Net for change detection — 50 epochs, lr=5e-5, Focal Loss
- [ ] Save best checkpoint (by validation F1) for each model
- [ ] Log metrics using standard print statements or lightweight logging

Phase 5 — Inference + Rule Engine (Week 4)

- [ ] Run both models on all 3 AOIs

- [ ] Apply GIS boundary mask — restrict outputs to factory polygon only
- [ ] Apply rule: if NDVI drops > 15% within factory boundary → flag MEDIUM; > 30% → HIGH
- [ ] Combine change detection mask + vegetation mask → final violation map

Phase 6 — Explainability + Report (Week 4)

- [ ] Apply GradCAM++ to segmentation model — highlight which pixels drove the prediction
- [ ] Annotate before/after imagery with violation bboxes using Pillow/Matplotlib
- [ ] Generate compliance report per factory — PDF with imagery, metrics, and violation flags
- [ ] Export violation polygons as GeoJSON/shapefile for QGIS overlay

---

⚠️ Important Things to Watch Out For

Data pitfalls

- **Cloud cover** — always filter Sentinel-2 to < 10% cloud. Use median composites over a season (not single dates) to avoid cloud contamination.
- **Seasonal NDVI variation** — vegetation naturally drops in dry season. Always compare T1 and T2 from the **same season** (e.g. both post-monsoon, both dry). Comparing Dec to Jun will generate false violations.
- **CRS mismatch** — KGIS shapefiles may come in different projections. Always reproject everything to EPSG:32643 before clipping.
- **Sentinel-2 harmonisation** — use `S2_SR_HARMONIZED` collection in GEE, not `S2_SR`. The harmonised collection fixes a processing baseline change in 2022 that would otherwise create false NDVI differences.

Model pitfalls

- **Class imbalance** — changed/violation pixels are rare (5–15% of total). Use Focal Loss, not plain BCE, for change detection. Report F1, not accuracy.
- **Overfitting** — with small datasets (< 300 patches), use heavy augmentation and dropout. Monitor validation F1 every epoch.
- **Resolution mismatch** — B11/B12 Sentinel-2 bands are 20m, not 10m. Upsample to 10m using bilinear interpolation before stacking with B4/B8.

QGIS tips

- Use the **Semi-Automatic Classification Plugin** in QGIS for supervised land cover classification — useful for quick validation

- Use **Temporal/Spectral Profile Tool** to inspect NDVI time series at specific pixel locations
- Always save your QGIS project with **relative paths** so collaborators can open it

Evaluation pitfalls

- Do not evaluate on the same factories you trained on — use a spatial train/test split (e.g. Peenya for training, Bommasandra for testing)
- Report Kappa alongside accuracy — with imbalanced classes, 90% accuracy is meaningless if 90% of pixels are unchanged
- For the GradCAM IoU metric, you need at least 20 manually annotated violation regions to compute a reliable score

---

🗂️ Annotated Reading List

Paper	Why read it
SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers (Xie et al., 2021)	Architecture of your segmentation model
Remote Sensing Image Change Detection with Transformers — BIT (Chen et al., 2021)	Architecture behind your change detection model
Deep Learning for Land Cover Change Detection (Zhu et al., 2017)	Foundational survey for your literature review
BigEarthNet: A Large-Scale Benchmark Archive (Sumbul et al., 2019)	Pretraining dataset reference
Grad-CAM: Visual Explanations from Deep Networks (Selvaraju et al., 2017)	Your explainability method
CPCB Guidelines on Green Belt Development	Legal context — cite this in your report

---

🔑 Quick Reference Card

```
Project     : EcoWatch AI
Problem     : Automated industrial green belt compliance monitoring
```

```
Location    : Peenya / Bommasandra / Nanjangud, Karnataka, India
Imagery     : Sentinel-2 SR Harmonised (GEE) — 10m resolution
GIS data    : KGIS industrial boundaries (kgis.ksrsac.in)
T1 period   : 2019 post-monsoon (Oct–Dec)
T2 period   : 2023 post-monsoon (Oct–Dec)
CRS         : EPSG:32643 (WGS84 UTM Zone 43N)
Patch size  : 512 × 512 pixels
Input bands : B4 (Red), B3 (Green), B2 (Blue), B8 (NIR), NDVI = 5 channels

Model 1     : SegFormer-B2        → vegetation segmentation
Model 2     : Siamese U-Net       → change detection
Model 3     : LSTM                → NDVI anomaly over time
Encoder     : ResNet-50 pretrained on ImageNet / BigEarthNet
Loss 1      : Dice + BCE (segmentation)
Loss 2      : Focal Loss (change detection, imbalanced)
Optimiser   : AdamW, lr = 1e-4 (seg), 5e-5 (change)

Key metric  : F1-change ≥ 0.70, mIoU-veg ≥ 0.75
Violation   : NDVI drop > 15% within factory boundary = MEDIUM risk
              NDVI drop > 30% within factory boundary = HIGH risk
XAI         : GradCAM++ for spatial evidence
Output      : PDF compliance report + annotated imagery + GeoJSON
```

EcoWatch AI · Master Reference · Keep this file in your repo root as `REFERENCE.md`